Project Documentation & Report

CSA2001: Fundamentals in AI and ML

Submitted To: Dr. Vinesh Kumar

Submitted By: Akshansh

Regd. No.: 25BCE11124

Contents

- **Introduction**
- **Objectives**
- **Scope**
- **Tools**
- **System Architecture**
- **Implementation**
- **Features**
- **Advantages**
- **Limitations**
- **Future Enhancements**
- **Learning & Key Takeaways**

Introduction

The BollywoodMotivationBot is an interactive, Python-based recommendation engine designed to provide users with cinematic inspiration from modern Indian cinema. Built using an object-oriented approach, the system serves as a specialized chatbot that suggests highly rated, motivational Bollywood films—primarily those released after 2017—based on the user's current emotional or professional needs.

Objectives

- To understand the basics of chatbot development
- To implement pattern matching using regular expressions
- To simulate human-like conversation using Python
- To build an interactive command-line chatbot

Scope of the Project

- Can be used as a beginner AI/ML project
- Helps understand NLP basics
- Can be extended into advanced AI moviebots
- Useful in customer support automation (basic level)

Tools and Technologies Used

- **Programming Language:** Python
- **Libraries Used:**
  - re (Regular Expressions)
  - random (Random response selection)
- **Platform:** Command Line / Terminal

Functional Requirements

Data Initialization and Management

- **System Startup**: The system must automatically load the movie and quote databases upon initialization of the bot object.
- **Database Categorization**: The system must store movie data categorized into specific motivational themes: struggle_to_success, career_ambition, sports_spirit, and life_lessons.
- **Metadata Storage**: For each movie, the system must maintain details including title, release year, IMDb rating, core theme, story summary, motivational message, key lessons, and iconic scenes.
- **Quote Repository**: The system must store a collection of iconic movie dialogues and general motivational "Power Quotes".

User Interaction and Input Processing

- **Interactive Interface**: The system must provide a command-line interface (CLI) that allows users to enter text-based queries.
- **Input Normalization**: The system must process user input by stripping whitespace and converting text to lowercase to ensure consistent matching.
- **Session Termination**: The system must recognize specific exit commands (e.g., "quit", "exit", "bye") to gracefully end the chat session.
- **History Tracking**: The system must record a history of all user queries and system responses during a single session.

Movie Search and Recommendation Logic

- **Keyword Detection**: The system must use regular expressions to identify specific keywords (e.g., "fail", "upsc", "sport") in the user's input.

- **Thematic Mapping**: If a keyword match is found, the system must select a movie from the corresponding category.
- **Random Selection**: If multiple movies exist in a category, or if no keyword match is found, the system must randomly select a film to provide variety.
- **Fallback Mechanism**: In cases where no specific keyword is detected, the system must provide a general movie recommendation from the entire database.

Response Generation and Output

- **Formatted Output**: The system must generate a structured "Motivation Card" for the user that includes the movie's title, rating, story, and why it is motivational.
- **Dynamic Quote Integration**: The system must attempt to pair the recommended movie with a specific quote from that film; otherwise, it must provide a general "Power Quote".
- **Category-Specific Mantras**: The system must append a unique "Today's Mantra" based on the category of the recommended movie (e.g., failure-themed for life_lessons).
- **Automated Recommendations**: The system must be capable of providing a "Top Pick" recommendation without requiring a specific user query.

Non Functional Requirements

Performance and Latency

The system must provide rapid feedback to ensure a smooth conversational experience:

- **Response Time**: The bot should generate a movie recommendation in under **200ms**, as the data retrieval involves local dictionary lookups rather than external API calls.
- **Initialization Speed**: The bot must load its internal databases (movies_db and quotes_db) instantly upon startup to minimize wait times for the user.

Usability and User Experience

The interface should be intuitive for non-technical users:

- **Clarity of Instructions**: The system must display a clear welcome message and example queries (e.g., "sports movies") to guide the user on how to interact with the bot.
- **Case Insensitivity**: The system must handle user inputs regardless of capitalization (e.g., "UPSC" vs "upsc") to ensure consistent behavior.
- **Graceful Exit**: Users must be able to terminate the session using common natural language commands like "bye" or "thanks".

Reliability and Robustness

The bot must be dependable even when user input is unpredictable:

- **Fallback Mechanism**: If the user's query does not match any specific keyword, the system must provide a random "general" recommendation rather than failing or returning an error.
- **Default Values**: If a specific quote for a movie is missing, the system must use a default "Power Quote" to maintain the motivational tone.

Maintainability and Readability

The codebase should be structured for easy updates by developers:

- **Type Safety**: The code uses Type Hinting (e.g., Dict, List, Optional) to ensure developers understand the expected data structures, reducing the likelihood of bugs during updates.
- **Modularity**: Movie data and quote data are isolated in specific methods (_load_movies and _load_quotes), allowing a developer to add new films without altering the core logic.

Portability and Scalability

- **Platform Independence**: Written in standard Python, the bot should be able to run on any operating system with a Python 3.x interpreter without requiring external dependencies.
- **Data Scaling**: While currently using a local dictionary, the architecture should be modular enough to eventually support an external JSON or SQL database as the movie count grows.

System Design

- ### Architecture

  The system architecture of the BollywoodMotivationBot is a monolithic, class-based design that follows a standard Request-Response pattern. It is structured into three logical layers: the Data Layer, the Logic Layer, and the Interface Layer, all encapsulated within a single Python class.

The Data Layer (Storage)

The "knowledge base" of the system is static and resides entirely within the class instance.

- **Movie Database (movies_db)**: A nested dictionary structure where movies are categorized by motivational themes (e.g., life_lessons, sports_spirit). Each entry contains metadata like IMDb ratings and core lessons.
- **Quote Database (quotes_db)**: A mapping of movie titles to their most iconic motivational dialogues.
- **State Tracking (history)**: A simple list that records every interaction, providing a basic foundation for session logging.

The Logic Layer (Processing)

This layer acts as the "brain" of the bot, translating user intent into specific data queries.

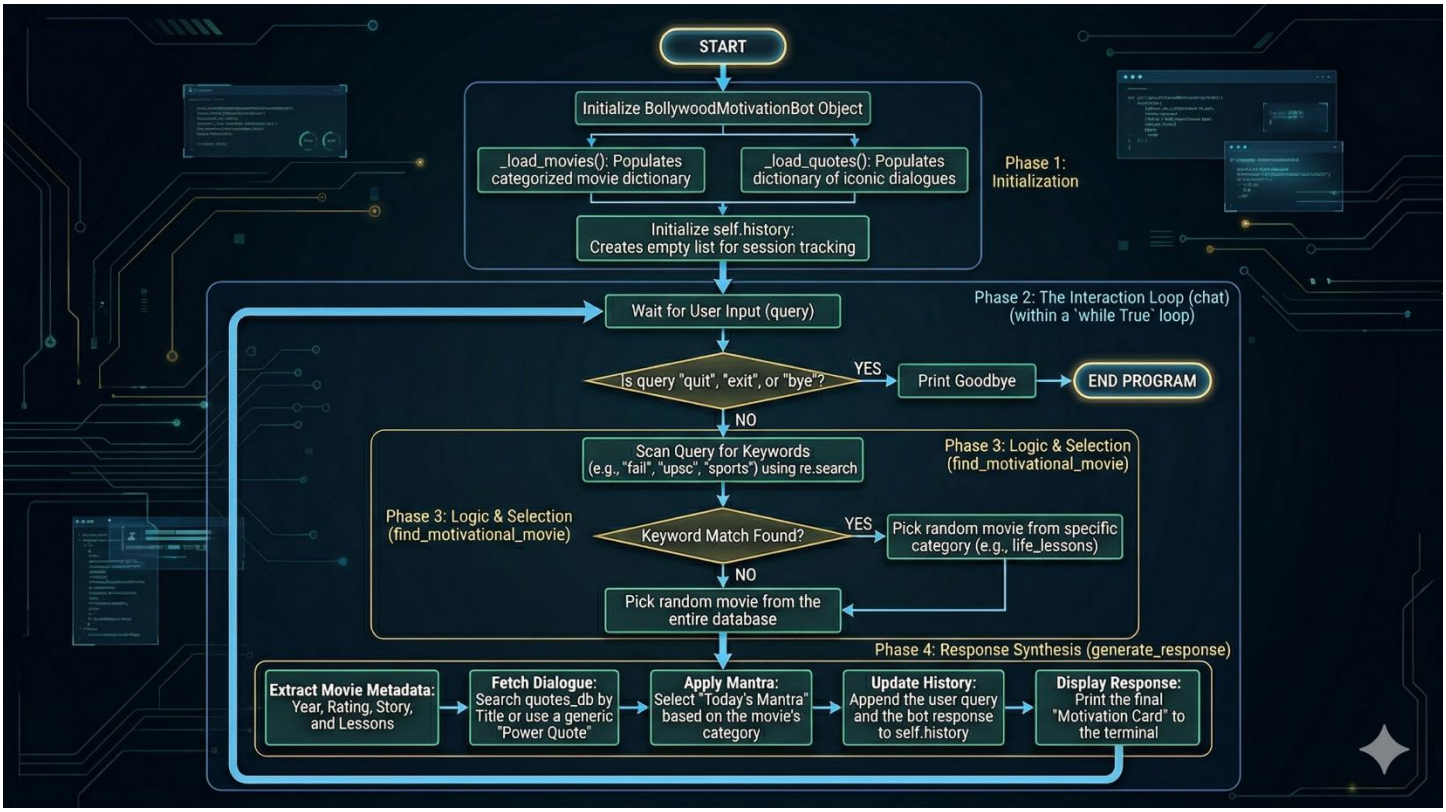
- **Keyword Engine (find_motivational_movie)**: This module uses the re (Regular Expression) library to perform pattern matching on user input. It maps common "pain points" (like "exam" or "failure") to the internal movie categories.
- **Randomization**: To ensure the bot doesn't feel repetitive, it uses random.choice() to select a specific movie or quote from the identified category.
- **Formatting Engine (generate_response)**: This component takes raw dictionary data and injects it into a pre-defined string template, adding dynamic elements like "Today's Mantra" based on the category.

The Interface Layer (User Interaction)

This is the outer shell that facilitates the dialogue.

- **Interaction Loop (chat)**: A while loop that captures standard input and prints the formatted response.
- **Exit Logic**: A set of termination triggers (e.g., "quit", "exit") that gracefully ends the session with a final motivational sign-off.

- **Logic found in Moviebot.py**:



Working Flow

1. User enters input
2. Input is converted to lowercase
3. Regular expressions match patterns
4. Corresponding response category is selected
5. Random response is generated
6. Output is displayed

Implementation Details

- ## Class Used

Internal State (Attributes)

The class maintains three primary attributes that define its "knowledge" and "memory":

- **self.movies_db**: A dictionary that stores all film details (ratings, stories, lessons) organized by themes like sports_spirit or life_lessons.
- **self.quotes_db**: A dictionary containing iconic dialogues used to enhance the motivational impact of the bot's responses.
- **self.history**: A list that records every query and response, allowing the bot to track the conversation flow.

Encapsulation Logic

The class uses **Encapsulation** by prefixing data-loading methods with an underscore (e.g., _load_movies). This signals that these are internal "helper" functions intended to be used by the class itself, while the public

methods like chat() or generate_response() are what the user interacts with.

Features

- Interactive chatbot
- Recognizes basic conversation patterns
- Randomized responses (more human-like)
- Easy to extend
- Lightweight and fast

Advantages

- Simple and beginner-friendly
- Fast execution
- Easy to modify

Limitations

Rigidity of Rule-Based Logic

The bot relies entirely on Hardcoded Keywords.

- **The Problem:** If a user types "I'm feeling uninspired by my exams," the bot will find "exam" and suggest *12th Fail*. However, if the user says "I'm struggling with my tests," the bot might fail to categorize it because "test" isn't in its dictionary.
- **Result:** The bot lacks Semantic Understanding (understanding meaning) and only performs String Matching.

Static Data Management

All the movie information is stored inside the script as a Python dictionary.

- **The Problem:** To add a new movie, you have to manually edit the source code. In a real-world scenario, this is inefficient.
- **Scalability:** If you wanted to include 1,000 movies, the code would become thousands of lines long and very difficult to manage.

Flaws in the Linear Regression Model

The integration of Linear Regression added earlier has specific mathematical limitations:

- **Oversimplification:** It assumes a direct mathematical relationship between the Release Year and the IMDb Rating. In reality, a movie released in 2024 isn't "naturally" better than one from 2016.
- **Low Feature Density:** The model only uses one "feature" (Year). Real rating predictions would require dozens of features like Director, Lead Actor, Budget, and Genre.
- **Assumption of Linearity:** Linear Regression tries to draw a straight line. However, movie quality/ratings usually fluctuate wildly and don't follow a straight upward or downward trend.

Lack of True Conversational Memory

While the code has a self.history list, it is functionally "blind" to previous turns.

- **The Problem:** If you say "Tell me about another sports movie," the bot doesn't "know" which one it just told you about. It treats every message as a brand-new start.

- **No Personalization:** It doesn't learn that you might hate wrestling movies but love cricket movies; it will keep suggesting *Dangal* or *Sultan* randomly.

No Error Handling or Edge Cases

The script is quite "brittle" when it comes to unexpected input:

- **Empty Queries:** If a user hits enter without typing, the logic might still try to process it.
- **Input Bias:** The find_motivational_movie function prioritizes the first keyword it finds. If you say "I failed my sports exam," it might give you a "Life Lesson" movie even if you wanted a "Sports" movie, depending on which keyword appears first in the dictionary.

Future Enhancements

Transition to Semantic Search (Vector Embeddings)

The current version uses regular expressions (re) for keyword matching, which fails if a user uses synonyms (e.g., "inspiring" instead of "motivational").

- **Embeddings:** You can use models like **Sentence-Transformers** (e.g., all-MiniLM-L6-v2) to convert movie descriptions into high-dimensional numerical vectors.
- **Similarity Matching:** Instead of keyword hits, the bot calculates the **Cosine Similarity** between the user's query vector and the movie vectors. This allows it to understand the *meaning* of a request, such as finding "underdog stories" even if that specific phrase isn't in the database.

Implementation of a Persistent Database

The movies_db is currently hardcoded as a dictionary, meaning data is lost if the script stops and is difficult to update.

- **SQL Integration:** Migrating to **SQLite** or **PostgreSQL** allows for structured storage of movie metadata, user history, and ratings.
- **Vector Databases:** Using specialized databases like **Pinecone**, **Milvus**, or **ChromaDB** enables the bot to perform sub-second similarity searches across thousands of movies.

Hybrid Recommendation Engines

A "Hybrid" approach combines multiple filtering techniques to provide more diverse and accurate suggestions.

- **Collaborative Filtering:** This analyzes patterns from multiple users to recommend movies that "people like you" also enjoyed.
- **Content-Based Filtering:** This continues to use the movie's specific attributes (genre, actor, theme) to find similar titles.
- **Cold-Start Mitigation:** Hybrid systems solve the "new user" problem by using content-based logic until enough user data is gathered for collaborative filtering.

Retrieval-Augmented Generation (RAG)

By integrating the bot with a Large Language Model (LLM) through RAG, you can ground the AI's responses in your verified movie data.

- **Accuracy:** This prevents the AI from "hallucinating" or making up movie details.
- **Conversational Depth:** The bot can answer complex, open-ended questions like "Which movie has a better father-daughter relationship: Dangal or Mimi?" by retrieving the relevant text segments before generating a response.

Advanced User Experience (UX) Features

Modern chatbots in 2026 focus on personalization and emotional intelligence.

- **Sentiment Analysis:** The bot can detect a user's emotional tone (frustrated, sad, or excited) and adjust its "Power Quotes" to be more empathetic or high-energy.
- **Explainable AI (XAI):** Adding "Why this movie?" sections fosters trust by explaining the logic behind a recommendation (e.g., "Recommended because you enjoyed the persistence theme in 12th Fail").
- **Watchlist Tracking:** Allowing the bot to remember "already seen" movies ensures it never recommends the same film twice.

Modules:

Regular Expressions (re)

This module is the "brain" behind the bot's keyword detection. It doesn't just look for exact matches; it uses patterns to find specific strings within the user's input.

- **Usage in code:** In find_motivational_movie, the script uses re.search(keyword, query_lower).

- **Why it's there:** It allows the bot to handle sentences. If you type "I want to watch a movie about **failing**," the re module identifies the root "fail" and correctly maps it to the life_lessons category.

Random (rand)

This module provides tools for generating non-deterministic outcomes.

- **Usage in code:** Used as random.choice(movies) or random.choice(quotes).
- **Why it's there:** To ensure the bot doesn't feel robotic. If there are five movies in the "Sports" category, random ensures you might get *83* the first time and *Paan Singh Tomar* the next. It also picks a random "Power Quote" to keep the motivation fresh.

typing

This is a support module used for **Type Hinting**. It doesn't change how the code runs, but it defines what the data *should* look like.

- **Usage in code:** Imports like Dict, List, Tuple, and Optional appear in function definitions (e.g., -> Dict[str, List[Dict]]).
- **Why it's there:** It acts as a manual for developers. It tells anyone reading the code exactly what kind of data is going into a function and what is coming out, which prevents bugs and makes the code much easier to maintain.

datetime

The datetime module is used for manipulating dates and times.

- **Usage in code:** It is imported at the top of the file (from datetime import datetime).
- **Why it's there:** Interestingly, in the current version of the script, this module is a "ghost import"—it is initialized but not actually used in the logic. It was likely included to eventually track the user's history with timestamps or to create a "Movie of the Day" feature.

numpy

If we look at the Linear Regression version, numpy is the industry standard for numerical computing.

- **Usage in code:** Used to structure data into arrays for the model.
- **Why it's there:** Machine Learning models cannot "read" lists the way humans do; they require data to be in a specific matrix or array format. numpy handles this conversion efficiently.

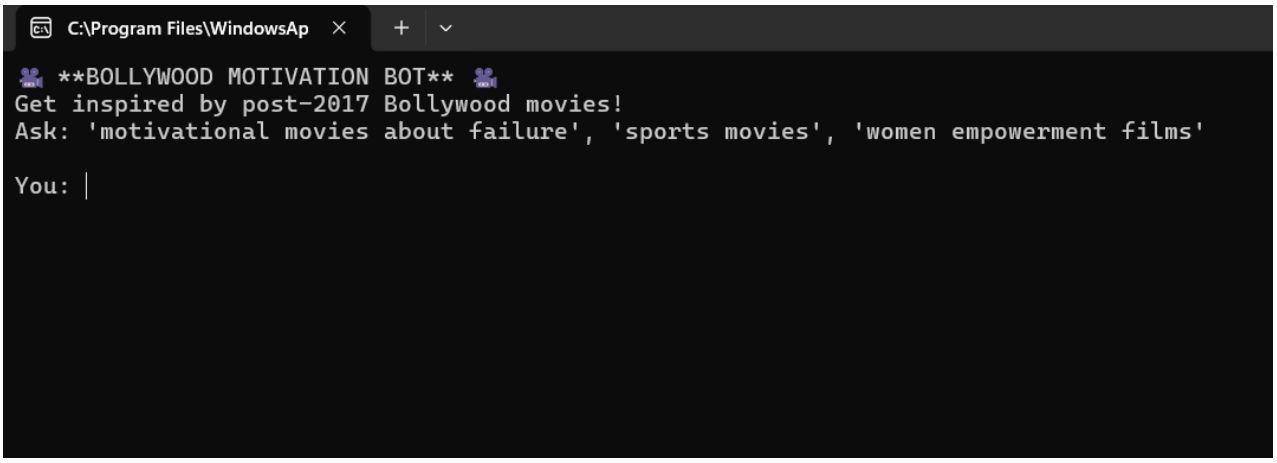
Functions:

- **__init__(self)**: This constructor initializes the bot by calling internal methods to populate the movie and quote databases. It also initializes an empty list called history to keep track of the conversation flow.
- **_load_movies(self)**: Acting as the primary data store, this method returns a dictionary containing details for motivational movies released after 2017 (with a few exceptions). The movies are organized into four categories: struggle_to_success, career_ambition, sports_spirit, and life_lessons. Each entry includes metadata like IMDb rating, core themes, story summaries, and specific lessons.

- **_load_quotes(self)**: This helper method returns a dictionary of iconic dialogues mapped to specific movie titles, such as *Dangal* and *12th Fail*. These quotes are used to add an extra layer of inspiration to the bot's responses.
- **find_motivational_movie(self, query)**: This is the logic engine of the bot. It takes a user's input string and uses regular expressions to scan for specific keywords like "fail," "career," or "upsc." If a match is found, it randomly selects a movie from the corresponding category; if no keywords match, it returns a random selection from the entire database.
- **generate_response(self, query)**: This function builds the final string displayed to the user. It retrieves a movie via find_motivational_movie, fetches a relevant quote, and formats the information into a structured template including the IMDb rating, "Quick Story," and "Why Watch." It also appends a category-specific "Mantra" for certain topics like life lessons or sports.
- **chat(self)**: This method manages the interactive terminal interface. It runs a while loop that captures user input, checks for exit commands (like "quit" or "bye"), and prints the bot's generated response for each query.

Result

Command Line Interface (CLI):



For Sports Movies:

```
C:\Program Files\WindowsAp

🎥 **BOLLYWOOD MOTIVATION BOT** 🎥
Get inspired by post-2017 Bollywood movies!
Ask: 'motivational movies about failure', 'sports movies', 'women empowerment films'

You: sport movies

🤖 MotivationBot:

 **Recommended Movie: Paan Singh Tomar (2012)* (2012)** 🎥

 **IMDb Rating:** 8.1/10
 **Core Message:** Channel your anger into excellence and world records.

 **Quick Story:**
Army runner becomes bandit after injustice.

 **Why Watch for Motivation:**
- Talent needs opportunity
- Injustice fuels determination
- Records > revenge

 **Iconic Scene:** Paan Singh's national record race
 **Power Quote:** Kamyabi pakad ke nahi aati, pakadni padti hai!

 **Where to Watch:** Netflix/Prime/OTT platforms


🏆 **Today's Mantra:** Teamwork makes the dream work!

What else motivates you? 🎯
====================================================================

You: |
```

For Woman Empowerment:

```
C:\Program Files\WindowsAp X  +  v

Get inspired by post-2017 Bollywood movies!
Ask: 'motivational movies about failure', 'sports movies', 'women empowerment films'

You: women empowerment films

🤖 MotivationBot:

 **Recommended Movie: Dangal (2016)* (2016)** 🎥

 **IMDb Rating:** 8.3/10
 **Core Message:** Proves that with determination and hard work, anyone can achieve greatness.

 **Quick Story:**
A father trains his daughters to become world-class wrestlers against all odds.

 **Why Watch for Motivation:**
- Persistence beats talent
- Family support is key
- Break stereotypes

 **Iconic Scene:** Final wrestling match - Geeta vs Kadam
 **Power Quote:** Kamyabi pakad ke nahi aati, pakadni padti hai!

 **Where to Watch:** Netflix/Prime/OTT platforms


What else motivates you? 🎯
==================================================================

You: |
```

For motivational movies about failure:

```
C:\Program Files\WindowsAp

You: motivational movies about failure

🤖 MotivationBot:

 **Recommended Movie: 12th Fail (2023) (2023)** 🎥

 **IMDb Rating:** 9.0/10
 **Core Message:** 12th fail doesn't mean life fail - persistence creates IPS officers.

 **Quick Story:**
Manoj Kumar Sharma's real-life journey from failure to IPS officer.

 **Why Watch for Motivation:**
- Small steps daily
- Failure builds character
- Honesty > shortcuts

 **Iconic Scene:** Manoj's 4th UPSC attempt success
 **Power Quote:** Kamyabi pakad ke nahi aati, pakadni padti hai!

 **Where to Watch:** Netflix/Prime/OTT platforms


💪 **Today's Mantra:** Failure is progress in disguise!

What else motivates you? 🎯
======================================================================

You:
```

Key Learning and Takeaways

Object-Oriented Programming (OOP) Mastery

The code demonstrates a clean implementation of OOP principles to manage complex state and logic.

- **Encapsulation**: By using a class structure, the bot bundles its data (movies_db) and its behaviors (find_motivational_movie) into a single unit.
- **State Management**: The __init__ method ensures the bot starts with a fresh history and fully loaded databases every time it is instantiated.
- **Internal Logic Protection**: The use of "private" naming conventions (e.g., _load_movies) signals which methods are internal helpers and which are public interfaces.

Practical String Manipulation & Regex

The project shows how to perform effective "Intent Recognition" without a heavy AI model.

- **Regular Expressions**: The use of re.search allows the bot to identify keywords like "upsc" or "fail" within longer sentences, making the interaction feel more natural than simple exact-match logic.
- **Input Normalization**: Converting queries to lowercase and stripping whitespace ensures the logic remains robust against varying user typing styles.

- **Dynamic Templating**: Using f-strings to assemble the "Motivation Card" allows for a highly structured and readable output that looks professional in a terminal environment.

Advanced Data Structuring

The way data is stored in this project is a lesson in organizing multi-dimensional information.

- **Nested Dictionaries**: The movies_db uses a dictionary of lists of dictionaries, which is the industry-standard way to represent JSON-like data in Python.
- **Rich Metadata**: Instead of just storing a title, the code stores ratings, themes, stories, and specific lessons, allowing the bot to provide high-value content rather than just a name.
- **Mapping Tables**: The bot uses a keyword-to-category mapping dictionary to efficiently route user requests to the correct data subset.

User Experience (UX) for CLI

Even within a text-based terminal, the code implements several "pro" UX features:

- **Fallback Logic**: The system is designed never to crash; if it doesn't understand a query, it defaults to a general recommendation rather than an error message.
- **Variability**: By using random.choice(), the bot ensures that repeated queries for the same topic provide different movies and quotes, preventing the experience from feeling "stale".
- **Session Tracking**: Maintaining a history list allows the developer to eventually implement features like "don't show me the same movie twice" or conversation logging.

Conclusion

The BollywoodMotivationBot serves as a practical and engaging example of how a structured, rule-based approach can transform a static movie list into a functional "digital mentor". By leveraging Python’s object-oriented principles, the code creates a cohesive environment where data, logic, and user interaction work together to provide targeted inspiration.